Student Task Planner

Assignment 1: Multi-Activity App with Intents and Menus

Project Explanation and Life Cycle Report

Detail	Information
Student name	Hasher Amin
Roll number	245423
Program and class	BS Information Technology (BS-IT)
Course	CIT-646 Mobile Application Development
Platform	Android, Kotlin, Android Studio, XML layouts
Submission date	Monday, 5 October 2026

1. Introduction

This document explains the work I did for Assignment 1 of the Mobile Application Development course. The assignment asked me to build a small Android application that moves between several screens, passes data from one screen to another, and offers actions through the three kinds of menus that Android provides. I named the application **Student Task Planner**. It lets a student write down the tasks of different courses, see them in a list, open the details of any single task, mark it as done, share it with a friend, or delete it.

In the sections below I first describe what the assignment required, then I explain how each screen of the app works, how the screens talk to each other through Intents, how the menus behave, and what design decisions I took to make the app easy to use. At the end I answer the four life cycle and design questions that belong to the report part of the assignment.

2. What the Assignment Asked For

The assignment is worth 20 marks. Eighteen marks are given for the working application (the second course learning outcome) and two marks are given for the short written report (the first course learning outcome). The main requirements can be grouped as follows.

- **Five activities.** The app must have a dashboard, a task list, a screen for adding a task, a screen for task details, and an About screen. Every activity has to be declared in the manifest file, otherwise Android will not be able to start it.
- **Eight navigation links.** Five of them are explicit Intents between my own screens and three are implicit Intents that ask other apps (sharing, email and browser) to do something. Some of the links must carry data, and two of them must return a result.

- **Three menu types.** An Options menu on the toolbar of the list screen, a Context menu that appears on long press of a list item, and a Popup menu opened from a three-dot icon on the detail screen.
- **One consistent design.** The same colors, the same toolbar and the same theme on every screen, with layouts that work in portrait and landscape and that scroll when the keyboard is open.
- **Good usability.** Touch targets of at least 48dp, hints on all inputs, content descriptions on images, priority shown with text as well as colour, validation of the title, and feedback messages after every action.
- **Clean coding rules.** The Activity Result API instead of the old startActivityForResult, constants for extra keys, a catch for ActivityNotFoundException, one statement per line, and no hard-coded text.

The tasks are stored in memory only, which means that they are kept in a shared list while the app is running and are not saved to a file or database.

3. Overview of the Application

When the app starts, the student sees the dashboard with the app name, a welcome line and the number of pending tasks. From there the student can open the task list or the About page. In the task list the student adds new tasks, taps a task to read its details, or long-presses a task to get quick actions. On the detail screen a three-dot icon opens a small menu from which the task can be shared, marked as done or deleted. When the student comes back to the list, the list is already updated and a short message confirms what happened.

I used a teal colour palette with an amber accent. Teal is calm and suits a study planner, and the amber colour is used for the pending task counter so that it draws attention without looking aggressive. The app logo is a white check mark inside a teal circle, which immediately suggests finished tasks.

4. The Five Screens

4.1 MainActivity (Dashboard)

This is the entry point of the app. It shows the logo, the title, the student name and a line such as "Pending tasks: 3". The number is refreshed every time the screen becomes visible again, so if the student marks a task as done in another screen and returns, the count is already correct. There are two large buttons: **My Tasks**, which opens the list screen and sends the student name along with it, and **About**, which opens the About screen.

4.2 TaskListActivity (Task List)

This screen shows all the tasks in a ListView. Each row displays the task title (crossed out when the task is done), the course name, the due date, the priority in words and the status, either Pending or Done. The priority is written in text and also coloured (red for high, orange for medium, green for low), so a person who cannot distinguish colors still understands it. When the list is empty, a friendly message tells the student to tap Add Task. The screen also has the toolbar with the Options menu, the student name as a subtitle, and a big Add Task button at the bottom.

4.3 AddTaskActivity (Add Task)

This is the form for a new task. It contains a title field, a course field, a RadioGroup for priority (High, Medium, Low), a due date field and the Save and Cancel buttons. The due date field does not need typing: when the student taps it, a calendar dialog opens, and the chosen date is written in the DD/MM/YYYY format. The title is compulsory. If it is empty, an error message appears on the field itself and the form is not submitted. The whole form is inside a ScrollView, so even in landscape mode or with the keyboard open every field can be reached.

4.4 TaskDetailActivity (Task Details)

This screen displays one task with a label above every value: title, course, priority, due date and status. At the top right there is a three-dot icon, which is a 48dp touch target with a content description for screen readers. Tapping it opens the Popup menu. If the student presses Back without choosing anything, the list simply stays unchanged.

4.5 AboutActivity (About)

The About page shows my name, roll number, class and the app version. It also has two buttons. **Email Me** opens the email app with my address and the subject "CIT-646 Assignment 1" already filled in. **Android Docs** opens the Android developer website in the browser.

5. Navigation and Intents

An Intent is a message that asks Android to start something. When the Intent names the exact class to start, it is called explicit. When it only describes an action, such as sending text or opening a web page, it is called implicit and Android finds a suitable app. The table below lists the eight links of the assignment and how I implemented each one.

No.	From to	Type	What is sent or returned
N1	A1 to A2	Explicit	The student name travels as the extra STUDENT_NAME and is shown as the subtitle of the list toolbar.
N2	A1 to A5	Explicit	Opens the About screen. No data is needed.
N3	A2 to A3	Explicit, result expected	Add button or Options menu starts the form. Save returns title, course, priority and due date with RESULT_OK. Cancel returns RESULT_CANCELED.

No.	From to	Type	What is sent or returned
N4	A2 to A4	Explicit	Tapping a row sends title, course, priority, due date, status and the position of the task in the list.
N5	A4 to A2	Result	Mark Done or Delete returns an action (DONE or DELETE) and the position, so the list knows which task to change.
N6	A4 to a sharing app	Implicit, ACTION_SEND	Sends a plain-text summary of the task as EXTRA_TEXT.
N7	A5 to email app	Implicit, ACTION_SENDTO	Uses a mailto: address with the subject CIT-646 Assignment 1.
N8	A5 to browser	Implicit, ACTION_VIEW	Opens https://developer.android.com.

5.1 How results are received

For N3 and N5 I used the Activity Result API, which means that the list screen registers a launcher with registerForActivityResult and handles the answer inside a callback. This is the modern replacement of startActivityForResult, which is deprecated. The advantage is that the code which starts the screen and the code which handles the answer sit next to each other, so the program is easier to read and to explain.

5.2 Constants for extra keys

All extra names, such as TITLE, COURSE, POSITION and ACTION, are defined once in a small file called Extras.kt. Both the sending and the receiving screen use the same constant. This prevents spelling mistakes, because a wrong key would compile without any warning but would silently return nothing at run time.

5.3 Safe implicit Intents

Sharing, email and browser Intents depend on apps that may not exist on a particular phone, for example an emulator without an email app. Every implicit Intent in my code is placed inside a try block and the ActivityNotFoundException is caught. If no app can handle the request, the user sees a short Toast message such as "No email app found", and the app continues to work normally instead of crashing.

5.4 The Back button

Every secondary screen has an arrow in the toolbar as well as the normal Back button of the phone. Both simply close the current activity and return to the previous one. Since no screen forces another screen to start on Back, there is no way to end up in a loop or a crash.

6. The Three Menus

6.1 Options Menu

The Options menu lives on the toolbar of the task list screen. It is defined in an XML file inside res/menu, and its items use titles from strings.xml. It contains the following items.

- **Add:** shown as a plus icon on the toolbar when there is room (ifRoom). It opens the Add Task form, the same as the big button.
- **Sort:** a sub-menu with three choices, By Priority, By Due Date and By Title. The list is reordered immediately and a message confirms it. Tasks without a due date are placed at the end when sorting by date.
- **About:** opens the About screen.
- **Exit:** always kept in the overflow menu (never) and closes the whole application using finishAffinity(), which finishes every activity of the app at once.

6.2 Context Menu

When the student presses and holds a task in the list, a Context menu appears. Its header shows the title of that task, and its items are View Details, Mark Done and Delete. I registered the ListView with registerForContextMenu and used AdapterContextMenuInfo to find out which row was pressed. Delete does not remove the task at once: an AlertDialog first asks "Are you sure you want to delete this task?", so an accidental long press cannot destroy data.

6.3 Popup Menu

On the detail screen the three-dot icon opens a Popup menu anchored to the icon itself. It offers Share, Mark Done and Delete. Share starts the implicit sharing Intent, while Mark Done and Delete close the screen and return the chosen action and the position to the list, which then updates the list and shows a confirmation message.

6.4 Rules followed in all menus

- Menu items are defined in XML with unique ids and with titles taken from strings.xml.
- Each handled item returns true, and the else branch calls the super method so that system items such as the back arrow still work.
- Every statement in the handlers is written on its own line, with no statements joined by semicolons.

7. How the Data Is Handled

A task is represented by a Kotlin data class named Task with five fields: title, course, priority, due date and done. All tasks are kept in one shared list inside an object called TaskRepository. Because it is an object, there is only one copy of the list, and every activity can read it without passing the

whole list around in Intents. This also means that the position number sent in N4 and N5 always points to the correct item in the same list.

As the assignment says, tasks live in memory only. They stay available while the app is open, even when the screen is rotated or when the student moves between screens, but they disappear when the app process is closed. I did not use files, SharedPreferences or a database, because the assignment explicitly limits storage to memory. For a real product the next step would be to save the tasks using Room or a similar library.

Priority is stored as a number (0 for High, 1 for Medium, 2 for Low). This makes sorting by priority a simple comparison, and the text shown on screen is taken from strings.xml, so no label is hard-coded.

8. User Interface and Design Decisions

8.1 Consistent theme

All colors are defined once in colors.xml and used through one app theme. The toolbar is written once in its own layout file (toolbar.xml) and is included on every screen, so it looks identical everywhere. Buttons share one style that gives them the same teal background, white text and a minimum height of 48dp.

8.2 Responsive layouts

I used LinearLayout and ScrollView so that no views overlap. Each screen puts its content in a ScrollView, which makes it usable in landscape mode and keeps the fields visible when the keyboard opens. The activities adapt to screen rotation, and because the task list is held outside the activities, rotating the phone does not lose any task. I also handled the system bars (status bar and navigation bar) with window insets so that on new Android versions the toolbar is never hidden behind the status bar.

8.3 Usability and accessibility

- **Touch targets:** buttons, radio buttons, text fields and the three-dot icon are at least 48dp high.
- **Hints and labels:** every input has a hint, and the radio group has a visible label.
- **Content descriptions:** the logo and the three-dot icon have descriptions for screen readers.
- **Text with colour:** the priority is written as High, Medium or Low in addition to its colour.

8.4 Validation and feedback

The title field uses setError to show "Title is required" directly on the field. After adding, finishing or deleting a task, a Toast message confirms the action, and a message appears when the list is empty. This way the user always knows what the app has done.

9. Project Structure

The project follows the standard Android Studio layout. The package name is com.example.hasheramin245423, which contains my name and roll number as the assignment demands. The table below explains the main files.

File	Purpose
AndroidManifest.xml	Declares the five activities, the launcher activity, the app icon and the theme.
MainActivity.kt	Dashboard. Shows the pending count and starts the list and About screens.
TaskListActivity.kt	Shows the list, receives results from the add and detail screens, and contains the Options and Context menus.
AddTaskActivity.kt	Form with validation and date picker. Returns the new task data.
TaskDetailActivity.kt	Shows one task, the Popup menu, and the sharing Intent.
AboutActivity.kt	Student information, email and browser Intents.
BaseActivity.kt	Common code for the toolbar, back arrow and system bar padding.
Task.kt	The Task data class, the shared TaskRepository and helper functions for priority and dates.
TaskAdapter.kt	Adapter that draws every row of the ListView.
Extras.kt	All Intent extra keys as constants.
res/layout	Screen layouts for the five activities, one list row layout and the shared toolbar.
res/menu	The three menu definitions (options, context and popup).
res/values	colors.xml, strings.xml and themes.xml, which hold all colors, texts and styles.
res/drawable	Vector icons for add, the three-dot menu and the logo.

10. How to Build and Run

The project is built with the latest stable Android Studio using Kotlin, AppCompat and a minimum SDK of 24 or higher. To run it, open the project, press Sync Project with Gradle Files, and click Run with a phone or emulator selected. To create the APK for submission, I use Build, then Build APK(s). Before zipping the project for submission, the build and .gradle folders are deleted so that the file stays small.

11. Testing

I tested the app on a real phone using the following scenarios. Each of them behaved as expected.

Test	Expected result
Open the app	Dashboard shows pending count 0 and both buttons.
Tap My Tasks	List opens with the student name as subtitle and the empty message.
Save a task with an empty title	Error appears on the title field and the form stays open.
Add a valid task	Returns to the list, the task appears, and a Toast says Task added.
Tap Cancel on the form	Returns to the list without adding anything.
Tap a task	Detail screen shows all fields correctly.
Popup menu, Share	Sharing sheet opens with the task summary.
Popup menu, Mark Done	List shows the task crossed out as Done; pending count decreases.
Popup menu, Delete	Task is removed and a confirmation message appears.
Long press a task	Context menu opens with the task title as header.
Context menu, Delete	Confirmation dialog appears before deletion.
Sort sub-menu	List reorders by priority, due date or title.
Exit in Options menu	All screens of the app close.
Email Me and Android Docs	Email app and browser open; a Toast is shown if none exists.
Rotate the phone on every screen	No crash, and tasks are still present.

12. Life Cycle and Design Questions

Question 1: Life-cycle callbacks when TaskListActivity starts AddTaskActivity

When the student taps Add Task, TaskListActivity first calls onPause. Then AddTaskActivity runs onCreate, onStart and onResume, and it becomes visible and ready for input. Since the form covers the whole screen, TaskListActivity finally calls onStop. When the student saves or cancels, AddTaskActivity calls onPause, TaskListActivity runs onRestart, onStart and onResume, and then AddTaskActivity runs onStop and onDestroy.

Question 2: The back stack, Back on TaskDetailActivity, and finishAffinity()

The activities of my app are kept in a stack, with the newest screen on top. When Back is pressed on TaskDetailActivity, that activity is removed from the top of the stack and destroyed, and TaskListActivity below it comes back to the front. When the Exit item calls finishAffinity(), Android

finishes the current activity and every activity below it that belongs to the same task, so the whole stack is cleared and the app closes completely instead of returning to the previous screen.

Question 3: Explicit versus implicit Intents

N1 to N5 move between screens that I wrote myself, so I know the exact class to start and an explicit Intent is the correct and safest choice. N6 to N8 need features that belong to other apps, such as sharing, email and a browser, and I cannot know which of those apps the user has installed, so I only describe the action and let Android choose through an implicit Intent. If no app can handle it, Android throws an ActivityNotFoundException; my app catches it and shows a Toast message such as "No email app found", so the app keeps working instead of crashing.

Question 4: User-interface principles applied

Consistency: every screen uses the same teal toolbar, the same button style and the same colour palette, so the student always feels at home. **Feedback:** the app answers every action, for example the Toast "Task added", the error on an empty title, and the confirmation dialog before a task is deleted. **Accessibility:** touch targets are at least 48dp, the icons have content descriptions, and the priority is written as text (High, Medium, Low) in addition to the colour, so nobody depends on colour alone.

13. Conclusion

Through this assignment I practised the most important basics of Android development: building several screens, moving between them with explicit and implicit Intents, sending data forward and getting results back with the Activity Result API, and offering actions through Options, Context and Popup menus. I also learnt how much small details matter, such as constants for extra keys, handling a missing app safely, validating input, and giving feedback to the user. The finished Student Task Planner meets every point of the requirements list and works in both portrait and landscape orientation. A natural improvement for the future would be to save the tasks permanently in a database and to add reminders before the due date.